



وظيفة البرمجة التفرعية

جامعة دمشق – كلية الهندسة المعلوماتية

مادة: الحقائق الافتراضية

العام الدراسي: 2025–2026

إعداد:

مجد خالد دباك 43274

محمد هاشم محمد برهان بدير 43338

أحمد خالد سليمان تلو 43023

حمزة علي الابراهيم 43111

فارس محمد نبيل الذهبي 43233

High-Performance Ticket Booking Engine

معلومات المشروع

- اسم المشروع: High-Performance Ticket Booking Engine
- نوع المشروع: نظام حجز تذاكر عالي الأداء
- التقنية المستخدمة: Laravel
- قاعدة البيانات: MySQL
- أدوات الاختبار: JMeter + Grafana

مقدمة

يهدف هذا المشروع إلى بناء نظام Back-End متخصص بمعالجة عمليات حجز التذاكر تحت الضغط العالي مع التركيز على:

- الأداء العالي
- معالجة التزامن Concurrency
- منع Race Conditions
- إدارة الموارد Resource Management
- توزيع الأحمال Load Distribution
- المعالجة الخلفية Background Processing
- تحسين قابلية التوسع Scalability

تم بناء النظام لمحاكاة سيناريوهات حقيقية مثل:

- حجز مقاعد السينما
- حجز المسرح
- حجز المحاضرات والفعاليات
- أنظمة التذاكر الإلكترونية

يركز المشروع على البنية الخلفية ومعالجة الضغط وليس على الواجهات الرسومية.

فكرة النظام

يقوم النظام بالسماح للمستخدمين بحجز مقاعد لفعاليات مختلفة. كل فعالية ترتبط بقاعة معينة، وكل قاعة تحتوي مجموعة من المقاعد. يقوم المستخدم بـ:

1. اختيار الفعالية
2. اختيار المقعد
3. تنفيذ عملية الحجز
4. التحقق من الرصيد
5. تنفيذ الدفع
6. تأكيد الحجز
7. إرسال إشعار

تم تصميم النظام لمحاكاة الضغط العالي عند محاولة عدة مستخدمين حجز نفس المقعد بنفس اللحظة.

المتطلبات الوظيفية الأساسية

يركز المشروع على عملية الحجز عالية التزامن وما يتعلق بها من عمليات دفع ومعالجة خلفية بأداء عالي وتحمل ضغط المستخدمين، ولا يهدف إلى بناء منصة إدارة متكاملة أو واجهات CRUD كاملة.

1- إنشاء عملية حجز

يقوم المستخدم بإرسال طلب حجز يتضمن:

- الفعالية
- المقعد

يقوم النظام بعدها بالتحقق من توفر المقعد.

2- التحقق من توفر المقعد

يقوم النظام بمنع حجز نفس المقعد أكثر من مرة لنفس الفعالية.

تم التركيز بشكل كبير على هذه النقطة بسبب مشاكل التزامن الناتجة عن الضغط العالي.

3- معالجة الدفع

بعد إنشاء الحجز المبدئي يقوم النظام بمحاكاة عملية دفع إلكترونية بالخلفية باستخدام Queue Jobs.

تشمل العملية:

- التحقق من الرصيد
- محاكاة بوابة الدفع
- احتمال فشل الدفع
- تحديث حالة الحجز

4- نظام المحفظة

يملك كل مستخدم رصيدًا داخل النظام.

يقوم النظام بـ:

- التحقق من الرصيد
- خصم قيمة التذكرة
- منع الحجز عند عدم كفاية الرصيد

5- تأكيد أو إلغاء الحجز

بعد انتهاء عملية الدفع:

- يتم تأكيد الحجز عند نجاح الدفع
- يتم إلغاء الحجز عند فشل الدفع
- يتم تحرير المقعد تلقائيًا عند الإلغاء

6- إرسال الإشعارات

تمت محاكاة إرسال إشعارات بالخلفية باستخدام Queue Jobs.

تشمل:

- إشعار نجاح الحجز
- إشعار فشل الدفع

7- إنشاء تقارير المبيعات اليومية

تم تنفيذ وظيفة خلفية تقوم بمعالجة بيانات الحجوزات اليومية وإنشاء تقرير مبيعات. تم تصميمها لمعالجة البيانات الكبيرة باستخدام Chunk Processing.

الكيانات الداعمة للنظام

يحتوي المشروع على عدة كيانات داعمة مستخدمة لخدمة عملية الحجز فقط مثل:

- Users المستخدمون
- Venues القاعات
- Seats المقاعد
- Events الفعاليات

هذه الكيانات لم يتم بناء CRUD APIs كاملة لها لأن المشروع يركز على:

- الأداء
- التزامن
- المعالجة الخلفية
- إدارة الموارد

وليس على بناء لوحة إدارة متكاملة.

تصميم قاعدة البيانات

users

يحتوي:

- بيانات المستخدم
- balance الرصيد

venues

يمثل القاعات أو المدرجات.

seats

يمثل المقاعد داخل القاعات

كل مقعد يحتوي:

- رقم
- القاعة المرتبط بها

events

يمثل الفعاليات.

كل فعالية تحتوي:

- الاسم
- الوصف
- وقت البداية
- وقت النهاية
- السعر
- القاعة المرتبطة

event_seats

جدول وسيط بين الفعاليات والمقاعد.

تم استخدامه للأسباب التالية:

- كل فعالية تملك حالة مستقلة لكل مقعد
- نفس المقعد يمكن استخدامه بفعاليات مختلفة
- يسمح بتخزين حالة المقعد:
 - Available
 - Reserved
 - Booked

هذا التصميم يمنع مشاكل التزامن بشكل أفضل.

bookings

يمثل الحجوزات.

كل حجز يحتوي:

- المستخدم
- event_seat
- حالة الحجز
- حالة الدفع
- وقت الحجز

daily_sales_reports

يستخدم لتخزين تقارير الجرد اليومية.

أدوات الاختبار والمراقبة

JMeter

تم استخدامه لمحاكاة:

- مئات وآلاف المستخدمين
- طلبات متزامنة
- الضغط العالي

تم استخدامه لقياس:

- زمن الاستجابة
- Throughput
- نسبة الأخطاء

Grafana

تم استخدامه لمراقبة استهلاك:

- المعالج
- RAM

Logs

تم استخدام Logging موسع لتتبع:

- الحجز
 - الدفع
 - الأخطاء
 - التضارب
 - معالجة ال Jobs
-

المشاكل التي تمت دراستها

Race Condition (1

وصف المشكلة

عند محاولة عدة مستخدمين حجز نفس المقعد بنفس اللحظة، قد يتم تنفيذ أكثر من عملية حجز ناجحة لنفس المقعد.

سبب المشكلة

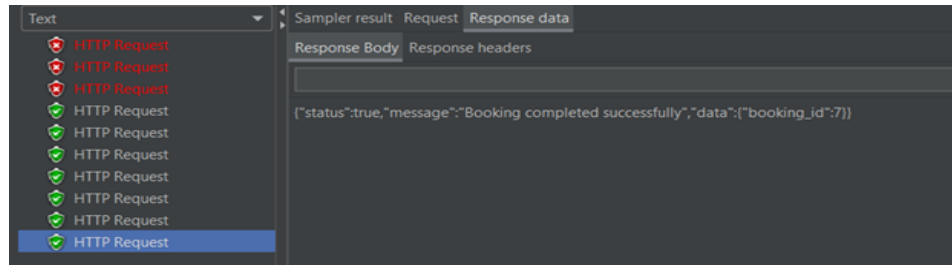
في البداية كان النظام يعتمد على:

1. التحقق من وجود حجز
2. ثم إنشاء الحجز

لكن عدة طلبات كانت تصل بنفس اللحظة قبل إنشاء الحجز الأول.

مما أدى إلى:

- نجاح عدة حجوزات لنفس المقعد
- خصم الرصيد أكثر من مرة وقد أصبح قيمته سالبة
- عدم اتساق البيانات



id	user_id	event_seat_id	status	payment_status	booked_at	created_at	updated_at
1	1	1	confirmed	paid	2026-05-16 12:07:25	2026-05-16 12:07:23	2026-05-16 12:07:25
2	1	1	confirmed	paid	2026-05-16 12:07:25	2026-05-16 12:07:23	2026-05-16 12:07:25
3	1	1	confirmed	paid	2026-05-16 12:07:25	2026-05-16 12:07:23	2026-05-16 12:07:25
4	1	1	confirmed	paid	2026-05-16 12:07:25	2026-05-16 12:07:23	2026-05-16 12:07:25
5	1	1	confirmed	paid	2026-05-16 12:07:26	2026-05-16 12:07:24	2026-05-16 12:07:26
6	1	1	confirmed	paid	2026-05-16 12:07:26	2026-05-16 12:07:24	2026-05-16 12:07:26
7	1	1	confirmed	paid	2026-05-16 12:07:26	2026-05-16 12:07:24	2026-05-16 12:07:26

id	name	email	email_verified_at	password	balance
1	Filomena Eichmann	jamison.quitzon@example.net	2026-05-16 12:06:42	\$2y\$12\$4WwkDuASgPJF1ShXXree.uUK8T1RHHzbRwtsi6tl6...	-125.00

حل المشكلة

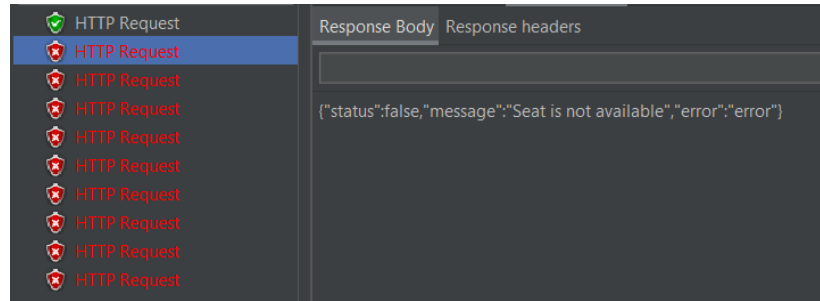
باستخدام الأقفال

آلية العمل

1. قفل السجل الخاص بالمستخدم باستخدام `User::lockForUpdate()`
2. إنشاء جدول `event_seat`
3. قفل السجل الخاص بالمقعد داخل الفعالية باستخدام `EventSeat::lockForUpdate()`
4. تغيير حالته من `Reserved` إلى `Available`

النتيجة

- ضمان عدم حجز نفس المقعد عدة مرات
- ضمان عدم خصم رصيد المستخدم عدة مرات أو الحصول على قيمة رصيد سالبة
- تصميم أكثر احترافية



2) ببطء عمليات الحجز

سبب المشكلة

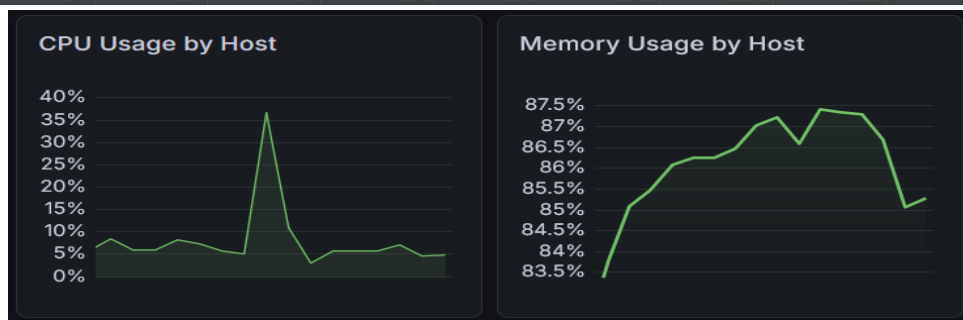
عملية الحجز كانت تتضمن داخل نفس الطلب:

- التحقق من المقعد
- إنشاء الحجز
- معالجة الدفع

مما أدى إلى:

- زمن استجابة مرتفع
- زيادة الضغط على السيرفر
- انخفاض Throughput

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	7844	1240	11192	1787.73	13.00%	8.2/sec	5.05	2.22	629.4
TOTAL	100	7844	1240	11192	1787.73	13.00%	8.2/sec	5.05	2.22	629.4



حل المشكلة

استخدام Queue + Background Jobs حيث تم فصل معالجة الدفع والإشعارات إلى Jobs مستقلة يتم تنفيذها بالخلفية

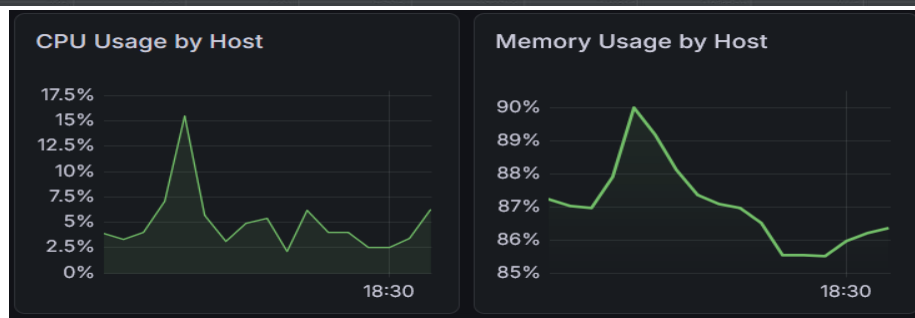
آلية العمل

1. الطلب الأساسي ينشئ الحجز
2. يتم وضع مهمة الدفع داخل Queue
3. يقوم Queue Workers بالمعالجة بالخلفية

النتيجة

- انخفاض زمن الاستجابة
- زيادة Throughput
- تحسين استهلاك الموارد

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	5180	891	8209	1828.53	1.00%	10.9/sec	4.46	2.93	419.3
TOTAL	100	5180	891	8209	1828.53	1.00%	10.9/sec	4.46	2.93	419.3



3) مشاكل معالجة البيانات الضخمة

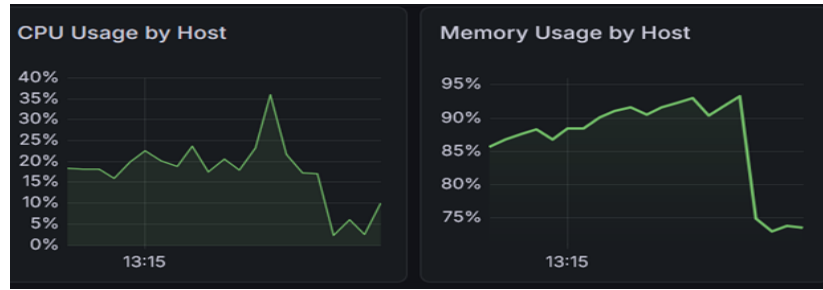
وصف المشكلة

عملية الجرد اليومية للمبيعات قد تتعامل مع آلاف أو ملايين الحجوزات مما يؤدي إلى:

- استهلاك كبير RAM
- بطء شديد
- احتمالية انهيار التطبيق

سبب المشكلة

قراءة جميع السجلات دفعة واحدة



```
PS D:\Projects\Laravel\ticket-booking-engine> php artisan queue:work
2026-05-17 10:13:17 App\Jobs\GenerateDailySalesReportJobV1 ..... RUNNING
PHP Fatal error: Allowed memory size of 2122317824 bytes exhausted (tried to allocate 16384 bytes) in D:\Projects\Laravel\ticket-booking-engine\vendor\laravel\framework\src\Illuminate\Database\Connection.php on line 425
PHP Fatal error: Allowed memory size of 2122317824 bytes exhausted (tried to allocate 65536 bytes) in D:\Projects\Laravel\ticket-booking-engine\vendor\laravel\framework\src\Illuminate\Foundation\Exceptions\Handler.php on line 445
```

حل المشكلة

تطبيق مبدأ Batch Processing باستخدام التعليمة:

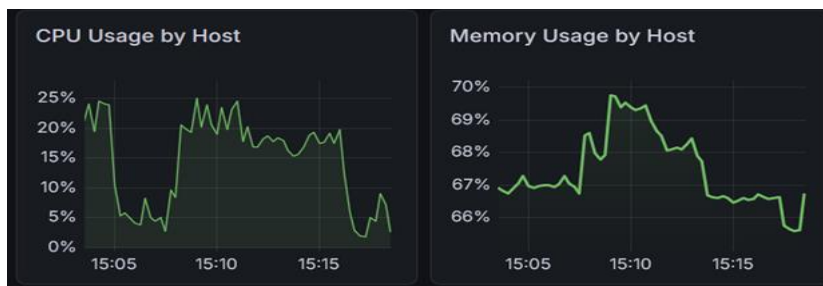
```
Booking::chunkById(1000, function ($bookings) {});
```

آلية العمل

1. تقسيم البيانات إلى دفعات صغيرة
2. معالجة كل دفعة بشكل مستقل

النتيجة

- استقرار أعلى
- استهلاك ذاكرة أقل



```
PS D:\Projects\Laravel\ticket-booking-engine> php artisan queue:work
2026-05-17 12:08:09 App\Jobs\GenerateDailySalesReportJobV2 ..... RUNNING
2026-05-17 12:16:10 App\Jobs\GenerateDailySalesReportJobV2 ..... 8m 1s DONE
```

4) إدارة الموارد الحاسوبية

وصف المشكلة

عند وجود عدد كبير من المستخدمين فإن استخدام Thread أو Worker واحد يؤدي إلى:

- عنق زجاجة
- تزداد مدة الانتظار
- يزداد الضغط على النظام

وعند استخدام عدد كبير منهم يؤدي لإستهلاك الموارد بشكل مفرض مما يؤثر بشكل سلبي على جودة النظام

حل المشكلة

Laravel لا تسمح بالتحكم بعدد Thread لذلك تمت محاكاة نفس الفكرة عن طريق استخدام: Multiple Queue Workers على عدة عمليات جرد (المستخدمة في الطلب السابق).

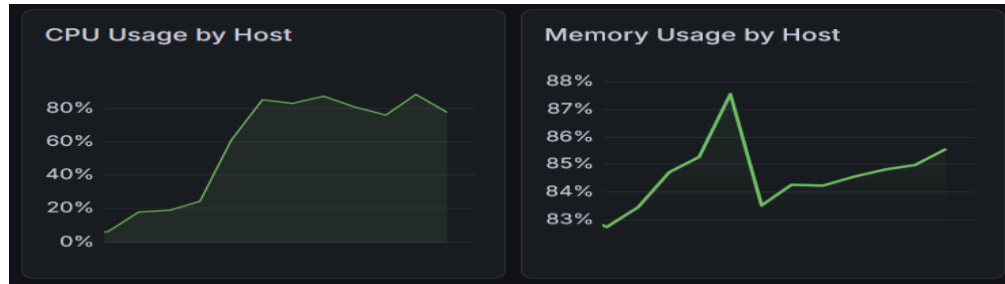
آلية العمل

تم تشغيل عدة Workers عن طريق تنفيذ التعليمة (php artisan queue:work) في عدة موجه أوامر.

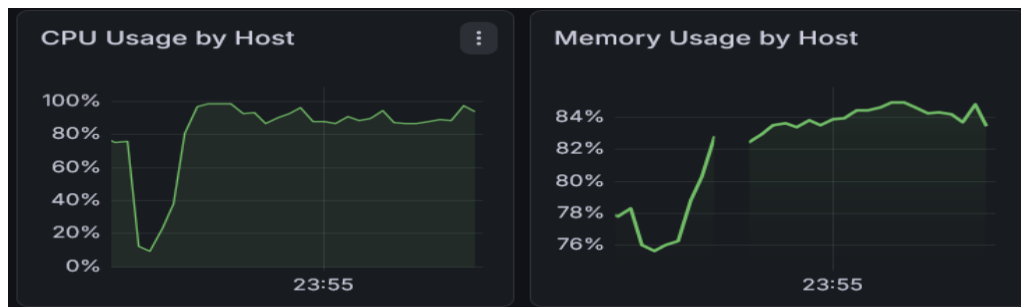
النتيجة

- معالجة jobs بالتوازي
- تحسين Throughput
- تقليل وقت الانتظار
- توزيع الحمل على عدة Workers

في المشكلة السابقة إستخدمنا worker واحد والآن سنجرب إستخدام 5 workers:



6 workers:



يمكننا ملاحظة أنه يمكننا إستخدام 5 عمال كحد أقصى

5) الضغط على الخادم

وصف المشكلة

وجود خادم واحد فقط يؤدي إلى:

- زيادة الضغط
- انخفاض الأداء
- ارتفاع نسبة الأخطاء

حل المشكلة

تم تطبيق عملية Load Balancer بإستخدام:

- Nginx
- Round Robin Strategy

لتوزيع الطلبات على عدة Laravel Instances حيث يختلف كل Instance يأخذ Port مختلف

استراتيجية التوزيع المستخدمة

تم اعتماد استراتيجية Round Robin والسبب:

- بسيطة
- عادلة
- منخفضة الكلفة الحسابية
- مناسبة للمشروع

النتيجة

- توزيع الحمل
- تحسين Throughput
- تقليل الضغط على خادم واحد
- زيادة الاستقرار

```
[2026-05-17 19:49:28] local.INFO: Server Port: 8000
[2026-05-17 19:49:47] local.INFO: Server Port: 8001
[2026-05-17 19:49:47] local.INFO: Server Port: 8002
[2026-05-17 19:49:48] local.INFO: Server Port: 8000
[2026-05-17 19:49:48] local.INFO: Server Port: 8001
[2026-05-17 19:49:48] local.INFO: Server Port: 8002
[2026-05-17 19:49:48] local.INFO: Server Port: 8000
[2026-05-17 19:49:48] local.INFO: Server Port: 8001
[2026-05-17 19:49:48] local.INFO: Server Port: 8002
[2026-05-17 19:49:48] local.INFO: Server Port: 8000
[2026-05-17 19:49:48] local.INFO: Server Port: 8001
[2026-05-17 19:52:16] local.INFO: Server Port: 8002
[2026-05-17 19:52:16] local.INFO: Server Port: 8000
[2026-05-17 19:52:16] local.INFO: Server Port: 8001
[2026-05-17 19:52:17] local.INFO: Server Port: 8002
[2026-05-17 19:52:17] local.INFO: Server Port: 8000
[2026-05-17 19:52:17] local.INFO: Server Port: 8001
[2026-05-17 19:52:17] local.INFO: Server Port: 8002
[2026-05-17 19:52:17] local.INFO: Server Port: 8000
[2026-05-17 19:52:17] local.INFO: Server Port: 8001
[2026-05-17 19:52:17] local.INFO: Server Port: 8002
```

(6) استراتيجية التخزين المؤقت الموزع (Distributed Caching)

وصف المشكلة

لوحظ أن بعض البيانات يتم طلبها بشكل متكرر من قبل المستخدمين، مثل الأحداث الأكثر حرجاً (Popular Events).

في التصميم التقليدي يتم تنفيذ استعلامات قاعدة البيانات في كل طلب، مما يؤدي إلى زيادة الحمل على قاعدة البيانات وارتفاع زمن الاستجابة عند ازدياد عدد المستخدمين.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	5120	4264	5875	406.18	2.00%	15.0/sec	20.90	3.03	1429.7
TOTAL	100	5120	4264	5875	406.18	2.00%	15.0/sec	20.90	3.03	1429.7

حل المشكلة

تم اعتماد Redis Cache باستخدام نمط (Cache Aside (Lazy Loading)، حيث يتم البحث عن البيانات في Redis أولاً، وفي حال عدم وجودها يتم جلبها من قاعدة البيانات ثم تخزينها في Redis لاستخدامها في الطلبات اللاحقة.

أماكن الإستخدام

عند الإستعلام عن:

- حدث معين.
- مقاعد حدث معين.
- الأحداث الأكثر حجراً.

النتيجة

- تقليل عدد الاستعلامات المرسلة إلى قاعدة البيانات.
- تحسين Throughput.
- تحسين زمن الاستجابة للطلبات المتكررة.
- تخفيف الحمل على الخادم.
- تحسين قدرة النظام على التعامل مع عدد أكبر من المستخدمين المتزامنين.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	2502	762	4277	1038.86	1.00%	19.1/sec	23.81	3.85	1278.6
TOTAL	100	2502	762	4277	1038.86	1.00%	19.1/sec	23.81	3.85	1278.6

(7) الأقفال الموزعة (Distributed Lock)

وصف المشكلة

بعد تحويل بعض العمليات إلى مهام خلفية (Background Jobs) تعمل بواسطة عدة Workers ، أصبح من الممكن أن تحاول عدة عمليات تنفيذ نفس المهمة في الوقت نفسه.

في حالة إنشاء تقرير المبيعات اليومية، قد يتم إرسال نفس المهمة أكثر من مرة أو قد يحاول أكثر من Worker تنفيذها بشكل متزامن، مما يؤدي إلى:

- تكرار تنفيذ نفس المهمة.
- استهلاك غير ضروري للموارد.
- احتمال إنشاء تقارير مكررة.
- زيادة الحمل على قاعدة البيانات.

حل المشكلة

تم اعتماد Distributed Lock باستخدام Redis عبر: `lock(...)` Cache قبل بدء إنشاء تقرير المبيعات اليومية.

عند بدء التنفيذ يحاول العامل الحصول على القفل، وإذا كان القفل مستخدماً من قبل عامل آخر يتم تجاهل التنفيذ أو الإنتظار حسب سياسة التنفيذ المعتمدة.

وبذلك نضمن أن عملية إنشاء التقرير تُنفذ مرة واحدة فقط مهما كان عدد العمال أو الخوادم العاملة في النظام.

8) ضمان سلامة المعاملات (ACID Transactions)

وصف المشكلة

تتكون عملية الحجز من عدة خطوات مترابطة، مثل:

1. إنشاء سجل الحجز.
2. التحقق من الرصيد.
3. خصم قيمة التذكرة من رصيد المستخدم.
4. تحديث حالة المقعد.
5. تأكيد الحجز.

في حال حدوث خطأ أثناء تنفيذ إحدى هذه الخطوات، قد تُنفذ بعض العمليات بينما تفشل عمليات أخرى، مما يؤدي إلى حالة غير متناسقة داخل قاعدة البيانات.

على سبيل المثال، قد يتم خصم الرصيد من المستخدم ثم يحدث خطأ قبل تأكيد الحجز، وبالتالي يخسر المستخدم أمواله دون الحصول على الحجز.

حل المشكلة

تم استخدام: `DB::transaction(...)` لتغليف جميع خطوات الحجز داخل معاملة واحدة.

وبذلك تصبح عملية الحجز عملية ذرية (Atomic)، حيث إما أن تنجح جميع الخطوات أو يتم التراجع عن جميع التعديلات تلقائيًا.

9 اختبار الإستقرار تحت الضغط

هدف الإختبار

يهدف هذا الاختبار إلى التحقق من قدرة النظام على التعامل مع عدد كبير من المستخدمين المتزامنين، وقياس أداء النظام تحت الضغط مع مراقبة زمن الاستجابة ومعدل معالجة الطلبات واستهلاك الموارد.

سيناريو الإختبار

تم إنشاء ملف بيانات يحتوي على مجموعة متنوعة من الطلبات المرسلة إلى واجهات النظام المختلفة، بحيث تحاكي سلوك المستخدمين الحقيقيين داخل النظام.

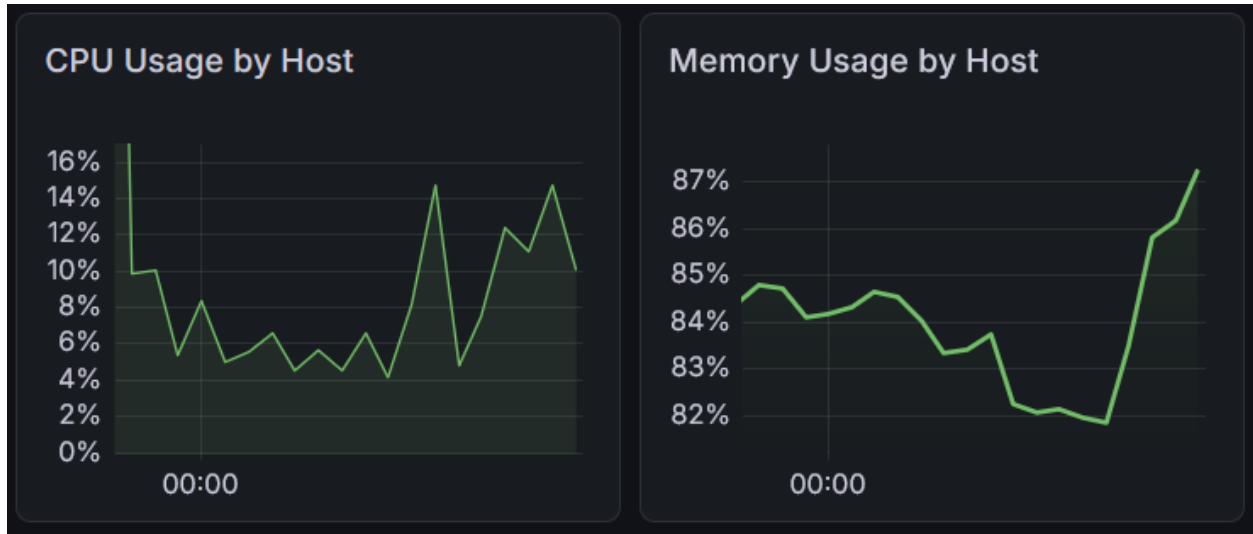
شملت العمليات المختبرة:

- عرض جميع الفعاليات.
- عرض تفاصيل فعالية محددة.
- عرض الفعاليات الأكثر طلباً.
- عرض المقاعد المتاحة لفعالية معينة.
- تنفيذ عمليات الحجز.

تم توزيع الطلبات بشكل عشوائي داخل ملف الاختبار بحيث لا يتم تنفيذ نوع واحد من العمليات فقط، وإنما محاكاة استخدام واقعي للنظام.

النتائج

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	250	10329	1481	19630	5395.66	7.20%	12.1/sec	203.98	3.24	17254.9
TOTAL	250	10329	1481	19630	5395.66	7.20%	12.1/sec	203.98	3.24	17254.9



تحليل النتائج

أظهرت النتائج قدرة النظام على التعامل مع عدد كبير من المستخدمين المتزامنين مع المحافظة على استقرار الخدمة.

كما أثبت الاختبار فعالية الحلول المستخدمة سابقاً مثل:

- معالجة الدفع باستخدام Queue و Background Jobs
 - استخدام آليات القفل لمنع التضارب أثناء الحجز.
 - استخدام التخزين المؤقت (Caching) لتخفيف الحمل على قاعدة البيانات.
- وقد ساهمت هذه الحلول في تحسين الأداء وتقليل تأثير الضغط الناتج عن زيادة عدد المستخدمين.

10) تحديد ومعالجة نقاط الاختناق

خلال مراحل تطوير المشروع واختبارات الضغط تم تحليل أداء النظام بهدف اكتشاف نقاط الاختناق التي تحد من قدرته على معالجة الطلبات المتزامنة.

وقد تم بالفعل توثيق إحدى أهم نقاط الاختناق في القسم الثاني من التقرير، حيث كانت عملية الحجز تتضمن تنفيذ معالجة الدفع ضمن نفس الطلب، مما أدى إلى زيادة زمن الاستجابة وانخفاض معدل معالجة الطلبات.

تمت معالجة هذه المشكلة باستخدام Queue و Background Jobs من خلال فصل عمليات الدفع عن الطلب الأساسي وتنفيذها في الخلفية بواسطة العمال (Workers)، الأمر الذي ساهم في تحسين الأداء وتقليل زمن الاستجابة وزيادة Throughput.

لذلك يمكن اعتبار متطلب تحليل نقاط الاختناق قد تم تحقيقه من خلال دراسة هذه المشكلة وتحليل أسبابها وتطبيق الحل المناسب لها ومقارنة النتائج قبل وبعد التحسين.

التقنيات المستخدمة

- Laravel
 - MySQL
 - Queue Workers
 - Laravel Jobs
 - Grafana
 - JMeter
 - Nginx
-

المراجع

- Laravel Documentation
- MySQL Documentation
- Nginx Documentation
- Apache JMeter Documentation